

HTTP Requests		
TOPIC: 01 Introduction 1.Laravel's Illuminate\Http\Request class provides an object-oriented way to interact with the current HTTP request being handled by your application as well as retrieve the input, cookies, and files that were submitted with the request.	<pre><?php namespace App\Http\Controllers; use Illuminate\Http\RedirectResponse; use Illuminate\Http\Request; class UserController extends Controller { /** * Update the specified user. */ public function update(Request \$request, string \$id): RedirectResponse { // Update the user... return redirect('/users'); } }</pre>	5.If you would like to get the current URL without a given query string parameter, you may utilize the fullUriWithoutQuery method: \$request->fullUriWithoutQuery(['type']); Retrieving the Request Host 1.You may retrieve the "host" of the incoming request via the host, httpHost, and schemeAndHttpHost methods: // http://localhost:8000 \$request->host(); // localhost \$request->httpHost(); // localhost:8000 \$request->schemeAndHttpHost(); // http://localhost:8000 Retrieving the Request Method 1.The method method will return the HTTP verb for the request. 2.You may use the isMethod method to verify that the HTTP verb matches a given string: \$method = \$request->method(); if (\$request->isMethod('post')) { // ... } Request Headers 1.You may retrieve a request header from the Illuminate\Http\Request instance using the header method. 2.If the header is not present on the request, null will be returned. 3.However, the header method accepts an optional second argument that will be returned if the header is not present on the request: \$value = \$request->header('X-Header-Name'); \$value = \$request->header('X-Header-Name', 'default'); 4.The hasHeader method may be used to determine if the request contains a given header: if (\$request->hasHeader('X-Header-Name')) { // ... } 5.For convenience, the bearerToken method may be used to retrieve a bearer token from the Authorization header. 6.If no such header is present, an empty string will be returned: \$token = \$request->bearerToken(); Request IP Address 1.The ip method may be used to retrieve the IP address of the client that made the request to your application: \$ipAddress = \$request->ip(); 2.If you would like to retrieve an array of IP addresses, including all of the client IP addresses that were forwarded by proxies, you may use the ips method. 3.The "original" client IP address will be at the end of the array: \$ipAddresses = \$request->ips(); 3.In general, IP addresses should be considered untrusted, user-controlled input and be used for informational purposes only.
TOPIC: 02 Interacting With The Request Accessing the Request 1.To obtain an instance of the current HTTP request via dependency injection, you should type-hint the Illuminate\Http\Request class on your route closure or controller method. 2.The incoming request instance will automatically be injected by the Laravel service container: <?php namespace App\Http\Controllers; use Illuminate\Http\RedirectResponse; use Illuminate\Http\Request; 5.class UserController extends Controller { /** * Store a new user. */ public function store(Request \$request): RedirectResponse { \$name = \$request->input('name'); // Store the user... return redirect('/users'); 5. } } 3.As mentioned, you may also type-hint the Illuminate\Http\Request class on a route closure. 4.The service container will automatically inject the incoming request into the closure when it is executed: use Illuminate\Http\Request; Route::get('/', function (Request \$request) { // ... }); Dependency Injection and Route Parameters 1.If your controller method is also expecting input from a route parameter you should list your route parameters after your other dependencies. 2.For example, if your route is defined like so: use App\Http\Controllers\UserController; Route::put('/user/{id}', [UserController::class, 'update']); 3.You may still type-hint the Illuminate\Http\Request and access your id route parameter by defining your controller method as follows:	Request Path, Host, and Method 1.The Illuminate\Http\Request instance provides a variety of methods for examining the incoming HTTP request and extends the Symfony\Component\HttpFoundation\Request class. 2.We will discuss a few of the most important methods below. Retrieving the Request Path 1.The path method returns the request's path information. 2.So, if the incoming request is targeted at http://example.com/foo/bar, the path method will return foo/bar: \$uri = \$request->path(); Inspecting the Request Path / Route 1.The is method allows you to verify that the incoming request path matches a given pattern. 2.You may use the * character as a wildcard when utilizing this method: if (\$request->is('admin/*')) { // ... } 3.Using the routels method, you may determine if the incoming request has matched a named route: if (\$request->routels('admin.*')) { // ... } Retrieving the Request URL 1.To retrieve the full URL for the incoming request you may use the url or fullUri methods. 2.The url method will return the URL without the query string, while the fullUri method includes the query string: \$url = \$request->url(); \$urlWithQueryString = \$request->fullUri(); 3.If you would like to append query string data to the current URL, you may call the fullUriWithQuery method. 4.This method merges the given array of query string variables with the current query string: \$request->fullUriWithQuery(['type' => 'phone']);	

HTTP response		
Topic: 01 : Creating Responses Strings and Arrays 1.All routes and controllers should return a response to be sent back to the user's browser. 2.Laravel provides several different ways to return responses. 3.The most basic response is returning a string from a route or controller. 4.The framework will automatically convert the string into a full HTTP response: Route::get('/', function () { return 'Hello World'; }); 5.In addition to returning strings from your routes and controllers, you may also return arrays. 6.The framework will automatically convert the array into a JSON response: Route::get('/', function () { return [1, 2, 3]; }); 7.Did you know you can also return Eloquent collections from your routes or controllers? 8.They will automatically be converted to JSON. 9.Give it a shot!	3.Or, you may use the withHeaders method to specify an array of headers to be added to the response: return response(\$content) ->withHeaders(['Content-Type' => \$type, 'X-Header-One' => 'Header Value', 'X-Header-Two' => 'Header Value',]); 4.You can remove specific headers from an outgoing response using the withoutHeader method: return response(\$content)->withoutHeader('X-Debug'); return response(\$content)->withoutHeader(['X-Debug', 'X-Powered-By']); Cache Control Middleware 1.Laravel includes a cache.headers middleware, which may be used to quickly set the Cache-Control header for a group of routes. 2.Directives should be provided using the "snake case" equivalent of the corresponding cache-control directive and should be separated by a semicolon. 3.If etag is specified in the list of directives, an MD5 hash of the response content will automatically be set as the ETag identifier: Route::middleware('cache.headers:public;max_age=30;s_maxage=300;stale_while_revalidate=600;etag')->group(function () { Route::get('/privacy', function () { // ... }); Route::get('/terms', function () { // ... }); });	Generating Cookie Instances 1.If you would like to generate a Symfony\Component\HttpFoundation\Cookie instance that can be attached to a response instance at a later time, 2.you may use the global cookie helper. 3.This cookie will not be sent back to the client unless it is attached to a response instance: \$cookie = cookie('name', 'value', \$minutes); return response('Hello World')->cookie(\$cookie); Expiring Cookies Early 1.You may remove a cookie by expiring it via the withoutCookie method of an outgoing response: return response('Hello World')->withoutCookie('name'); 2.If you do not yet have an instance of the outgoing response, you may use the Cookie facade's expire method to expire a cookie: Cookie::expire('name'); Cookies and Encryption 1.By default, thanks to the Illuminate\Cookie\Middleware\EncryptCookies middleware, all cookies generated by Laravel are encrypted and signed so that they can't be modified or read by the client. 2.If you would like to disable encryption for a subset of cookies generated by your application, you may use the encryptCookies method in your application's bootstrap/app.php file: ->withMiddleware(function (Middleware \$middleware): void { \$middleware->encryptCookies(except: ['cookie_name',]); }); 3.In general, cookie encryption should never be disabled, as this exposes your cookies to potential client-side data exposure and tampering.
Eloquent Models and Collections 1.You may also return Eloquent ORM models and collections directly from your routes and controllers. 2.When you do, Laravel will automatically convert the models and collections to JSON responses while respecting the model's hidden attributes: use App\Models\User; Route::get('/user/{user}', function (User \$user) { return \$user; });	Attaching Cookies to Responses 1.You may attach a cookie to an outgoing Illuminate\Http\Response instance using the cookie method. 2.You should pass the name, value, and the number of minutes the cookie should be considered valid to this method: return response('Hello World')->cookie('name', 'value', \$minutes); 3.The cookie method also accepts a few more arguments which are used less frequently. 4.Generally, these arguments have the same purpose and meaning as the arguments that would be given to PHP's native setcookie method: return response('Hello World')->cookie('name', 'value', \$minutes, \$path, \$domain, \$secure, \$httpOnly); 5.If you would like to ensure that a cookie is sent with the outgoing response but you do not yet have an instance of that response, you can use the Cookie facade to "queue" cookies for attachment to the response when it is sent. 6.The queue method accepts the arguments needed to create a cookie instance. 7.These cookies will be attached to the outgoing response before it is sent to the browser: use Illuminate\Support\Facades\Cookie; Cookie::queue('name', 'value', \$minutes);	Topic : 02 : Redirects 1.Redirect responses are instances of the Illuminate\Http\RedirectResponse class, and contain the proper headers needed to redirect the user to another URL. 2.There are several ways to generate a RedirectResponse instance. 3.The simplest method is to use the global redirect helper: Route::get('/dashboard', function () { return redirect('/home/dashboard'); }); 4.Sometimes you may wish to redirect the user to their previous location, such as when a submitted form is invalid. 5.You may do so by using the global back helper function. 6.Since this feature utilizes the session, make sure the route calling the back function is using the web middleware group: Route::post('/user/profile', function () { // Validate the request... return back()->withInput(); });

Asset Bundling (Vite)		
TOPIC: 01 : Introduction 1.Vite is a modern frontend build tool that provides an extremely fast development environment and bundles your code for production. 2.When building applications with Laravel, you will typically use Vite to bundle your application's CSS and JavaScript files into production-ready assets. 3.Laravel integrates seamlessly with Vite by providing an official plugin and Blade directive to load your assets for development and production.	}, }); 5.If you are building an SPA, including applications built using Inertia, Vite works best without CSS entry points: import { defineConfig } from 'vite'; import laravel from 'laravel-vite-plugin'; export default defineConfig({ plugins: [laravel({ 'resources/css/app.css', 'resources/js/app.js', }),], }); 6.Instead, you should import your CSS via JavaScript. Typically, this would be done in your application's resources/js/app.js file: import './bootstrap'; import './css/app.css'; 7.The Laravel plugin also supports multiple entry points and advanced configuration options such as SSR entry points. Working With a Secure Development Server6 1.If your local development web server is serving your application via HTTPS, you may run into issues connecting to the Vite development server. 2.If you are using Laravel Herd and have secured the site or you are using Laravel Valet and have run the secure command against your application, the Laravel Vite plugin will automatically detect and use the generated TLS certificate for you. 3.If you secured the site using a host that does not match the application's directory name, you may manually specify the host in your application's vite.config.js file: import { defineConfig } from 'vite'; import laravel from 'laravel-vite-plugin'; export default defineConfig({ plugins: [laravel({ // ... detectTls: 'my-app.test', }),], });	4.When using another web server, you should generate a trusted certificate and manually configure Vite to use the generated certificates: // ... import fs from 'fs'; const host = 'my-app.test'; export default defineConfig({ // ... server: { host, hmr: { host }, https: { key: fs.readFileSync('/path/to/\${host}.key'), cert: fs.readFileSync('/path/to/\${host}.crt'), }, }, }); 5.If you are unable to generate a trusted certificate for your system, you may install and configure the @vitejs/plugin-basic-ssl plugin. 6.When using untrusted certificates, you will need to accept the certificate warning for Vite's development server in your browser by following the "Local" link in your console when running the npm run dev command. Running the Development Server in Sail on WSL2 1.When running the Vite development server within Laravel Sail on Windows Subsystem for Linux 2 (WSL2), you should add the following configuration to your vite.config.js file to ensure the browser can communicate with the development server: // ... export default defineConfig({ // ... server: { hmr: { host: 'localhost', }, }, }); 2.If your file changes are not being reflected in the browser while the development server is running, you may also need to configure Vite's server.watch.usePolling option.
TOPIC: 02 : Installation & Setup 1.The following documentation discusses how to manually install and configure the Laravel Vite plugin. 2.However, Laravel's starter kits already include all of this scaffolding and are the fastest way to get started with Laravel and Vite.		
Installing Node 1.You must ensure that Node.js (16+) and NPM are installed before running Vite and the Laravel plugin: node -v npm -v 2.You can easily install the latest version of Node and NPM using simple graphical installers from the official Node website. 2.Or, if you are using Laravel Sail, you may invoke Node and NPM through Sail: ./vendor/bin/sail node -v ./vendor/bin/sail npm -v		
Installing Vite and the Laravel Plugin 1.Within a fresh installation of Laravel, you will find a package.json file in the root of your application's directory structure. 2.The default package.json file already includes everything you need to get started using Vite and the Laravel plugin. 3.You may install your application's frontend dependencies via NPM: npm install		
Configuring Vite 1.Vite is configured via a vite.config.js file in the root of your project. 2.You are free to customize this file based on your needs, and you may also install any other plugins your application requires, such as @vitejs/plugin-react, @sveltejs/vite-plugin-svelte or @vitejs/plugin-vue. 3.The Laravel Vite plugin requires you to specify the entry points for your application. 4.These may be JavaScript or CSS files, and include preprocessed languages such as TypeScript, JSX, TSX, and Sass. import { defineConfig } from 'vite'; import laravel from 'laravel-vite-plugin'; export default defineConfig({ plugins: [laravel({ 'resources/css/app.css', 'resources/js/app.js', }),], });		